



Análise Exploratória de Dados com Pandas



sort

pandas DataFrame			
nome	idade	salario	departamento
Ana	28	4500.00	Dados
Bruno	34	7200.50	Engenharia
Carla	29	6300.00	Dados
Daniel	41	9100.00	Marketing
Eva	26	3800.00	Dados
Felipe	33	6800.00	Engenharia
Gabriela	Null	Null	Null
Hugo	31	5500.00	Dados



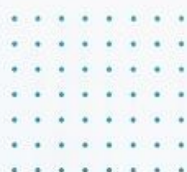
filter



groupby



limpeza de dados
(tratamento de nulos)



Guia Prático Completo de Manipulação de DataFrames em Python

Autor: Joaquim José Barão Perez Junior

Ano: 2026

1. Introdução

Este documento apresenta um guia completo e comentado de Análise Exploratória de Dados (EDA) utilizando a biblioteca Pandas do Python. O código original fornecido foi revisado, corrigido (erros de sintaxe e tipografia), organizado em seções lógicas e complementado com dados de exemplo para que qualquer pessoa possa executar e entender cada etapa.

A Análise Exploratória de Dados é a etapa fundamental de qualquer projeto de Data Science. Ela permite compreender a estrutura, qualidade e padrões dos dados antes de aplicar modelos mais complexos.

Bibliotecas necessárias: pandas (manipulação de dados), numpy (opcional para valores nulos).

2. Configuração e Dados de Exemplo

Para tornar este material 100% reproduzível, criamos DataFrames de exemplo em vez de depender de arquivos CSV externos. Você pode substituir facilmente pelos seus próprios arquivos.

2.1 Importação das bibliotecas

```
import pandas as pd
import numpy as np
```

2.2 Criação de dados de exemplo (funcionários)

Em um cenário real você carregaria com `pd.read_csv()` ou `pd.read_excel()`. Aqui geramos dados fictícios de funcionários:

```
# Dados de exemplo de funcionários
dados_funcionarios = {
    'id': [1, 2, 3, 4, 5, 6, 7, 8],
    'nome': ['Ana Silva', 'Bruno Costa', 'Carla Mendes', 'Diego Souza',
            'Elena Rocha', 'Felipe Lima', 'Gabriela Alves', 'Hugo Martins'],
    'idade': [28, 35, 42, np.nan, 31, 29, 45, 38],
    'salario': [4500, 7200, 9800, 5100, 6300, 4800, 11000, 8500],
    'id_departamento': [1, 2, 1, 3, 2, 1, 3, 2],
    'ativos': [True, True, True, False, True, True, True, False]
}
df = pd.DataFrame(dados_funcionarios)
print(df)
```

2.3 Dados de departamentos

```
dados_dept = {
```

```
'id_departamento': [1, 2, 3],
'nome_departamento': ['TI', 'Vendas', 'RH']
}
df_dept = pd.DataFrame(dados_dept)
print(df_dept)
```

2.4 Novos funcionários (para concatenação)

```
dados_novos = {
'id': [9, 10],
'nome': ['Isabela Freitas', 'João Pereira'],
'idade': [27, 33],
'salario': [5200, 6900],
'id_departamento': [1, 2],
'ativos': [True, True]
}
df_novos = pd.DataFrame(dados_novos)
```

3. Carregamento de Dados (Cenário Real)

Quando você tiver arquivos reais, use as funções abaixo. Os comentários originais foram corrigidos:

```
# Carregar CSV
df = pd.read_csv('nome_do_arquivo.csv')

# Carregar Excel (especifique a aba se necessário)
df = pd.read_excel('nome_do_arquivo.xlsx', sheet_name='nome_da_aba')
```

Observação: substitua 'nome_do_arquivo.csv' pelo caminho real do seu arquivo.

4. Seleção de Colunas e Visualização

4.1 Selecionar colunas específicas

```
# Mostra apenas as colunas 'nome' e 'idade'
df[['nome', 'idade']]

# Mostra apenas a coluna 'idade' (retorna uma Series)
df['idade']
```

Explicação: O uso de colchetes duplos [[]] retorna um DataFrame. Colchetes simples [] retornam uma Series (vetor unidimensional).

5. Filtros e Máscaras Booleanas

Filtros são uma das operações mais poderosas do Pandas. Eles criam máscaras True/False e selecionam apenas as linhas que satisfazem a condição.

5.1 Filtro simples

```
# Cria uma Series booleana (True/False)
```

```
filtro_maior_que_30 = df['idade'] > 30
print(filtro_maior_que_30)

# Aplica o filtro: mostra apenas linhas onde idade > 30
df[filtro_maior_que_30]

# Forma direta (equivalente)
df[df['idade'] > 30]
```

5.2 Combinação de filtros

```
filtro_maior_que_30 = df['idade'] > 30
filtro_mais_de_5mil = df['salario'] > 5000

# Operador & (AND) – ambas as condições verdadeiras
df[filtro_maior_que_30 & filtro_mais_de_5mil]

# Operador | (OR) – pelo menos uma verdadeira
df[filtro_maior_que_30 | filtro_mais_de_5mil]
```

Importante: use & e | (não 'and'/'or') e coloque cada condição entre parênteses quando escrever na mesma linha.

6. Criação e Remoção de Colunas

```
# Cria nova coluna 'salario_anual'
df['salario_anual'] = df['salario'] * 12
print(df)
```

```
# Remove a coluna criada (axis=1 significa coluna)
df = df.drop('salario_anual', axis=1)
# Alternativa moderna:
df = df.drop(columns=['salario_anual'])
```

Explicação: axis=0 remove linhas; axis=1 remove colunas. O método drop retorna um novo DataFrame, por isso reatribuímos a df.

7. Resumo dos Dados

7.1 Informações gerais (info)

```
df.info()
```

Mostra: número de linhas e colunas, nomes das colunas, quantidade de valores não-nulos e tipos de dados (dtypes). Essencial para detectar problemas de qualidade.

7.2 Estatísticas descritivas (describe)

```
df.describe()
```

Gera: contagem, média, desvio padrão, mínimo, quartis (25%, 50%, 75%) e máximo das colunas numéricas. A mediana é o valor 50%.

7.3 Estatísticas individuais

```
df['salario'].sum()    # Soma total dos salários
df['salario'].mean()   # Média aritmética
df['salario'].median() # Mediana (valor central)
df['salario'].max()    # Maior salário
df['salario'].min()    # Menor salário
df['salario'].std()    # Desvio padrão (complemento útil)
```

8. Tratamento de Valores Nulos

Valores nulos (NaN) são comuns em dados reais e podem distorcer análises.

```
# Remove TODAS as linhas que contenham qualquer valor nulo
df_limpo = df.dropna()

# Remove apenas linhas com nulos na coluna 'idade'
df_sem_nulos_idade = df.dropna(subset=['idade'])

# Preenche nulos com um valor (ex: média da idade)
df['idade'] = df['idade'].fillna(df['idade'].mean())
```

Dica: sempre analise quantos nulos existem com `df.isnull().sum()` antes de decidir o tratamento.

9. Conversão de Tipos

```
# Transforma float em inteiro (após tratar nulos)
df['idade'] = df['idade'].astype(int)

# Outros exemplos comuns:
# df['data'] = pd.to_datetime(df['data'])
# df['categoria'] = df['categoria'].astype('category')
```

10. Contagem de Valores

```
# Conta quantas vezes cada valor aparece na coluna 'ativos'
df['ativos'].value_counts()

# Com porcentagem
df['ativos'].value_counts(normalize=True) * 100
Útil para variáveis categóricas (sim/não, departamentos, etc.).
```

11. Agrupamento (GroupBy)

O `groupby` é o equivalente ao `GROUP BY` do SQL. Permite calcular estatísticas por grupos.

```
# Agrupa por departamento e calcula a média salarial
df.groupby('id_departamento')['salario'].mean()

# Soma dos salários por departamento
df.groupby('id_departamento')['salario'].sum()

# Maior salário por departamento
df.groupby('id_departamento')['salario'].max()

# Múltiplas agregações de uma vez
```

```
df.groupby('id_departamento')['salario'].agg(['mean', 'sum', 'max', 'count'])
```

Explicação: groupby() cria grupos; em seguida aplicamos uma função de agregação (mean, sum, max, etc.) na coluna desejada.

12. Ordenação

```
# Ordena por salário em ordem crescente (padrão)
df.sort_values(by='salario')
```

```
# Ordena por salário em ordem decrescente
df.sort_values(by='salario', ascending=False)
```

```
# Ordena por múltiplas colunas
df.sort_values(by=['id_departamento', 'salario'], ascending=[True, False])
```

Correção do código original: o parâmetro correto é ascending (não 'acscending').

13. Junção de Tabelas (Merge)

O merge é o equivalente ao JOIN do SQL. Combina DataFrames com base em colunas-chave.

```
# Quando as colunas-chave têm o MESMO nome
df_final = pd.merge(df, df_dept, on='id_departamento')
print(df_final)
```

```
# Quando as colunas-chave têm NOMES DIFERENTES
df_final = pd.merge(df, df_dept,
                    left_on='id_departamento',
                    right_on='id_departamento')
```

```
# Tipos de join: how='inner' (padrão), 'left', 'right', 'outer'
```

Correções: 'righ_on' → right_on; espaços extras nos nomes das colunas foram removidos.

14. Concatenação de Tabelas

Concatena DataFrames um embaixo do outro (ou lado a lado).

```
# Empilha as tabelas verticalmente e reinicia o índice
df_todos = pd.concat([df, df_novos], ignore_index=True)
print(df_todos)
```

ignore_index=True faz o índice recomeçar do zero. Sem ele, os índices originais são mantidos (pode haver duplicatas).

15. Exportação dos Resultados

```
# Salva em CSV
df_todos.to_csv('todos_funcionarios.csv', index=False)
```

```
# Salva em Excel
df_todos.to_excel('todos_funcionarios.xlsx', index=False)
```

index=False evita que o índice do DataFrame seja salvo como uma coluna extra. Correção: faltava aspas de fechamento no código original.

16. Boas Práticas e Dicas

- Sempre use `df.copy()` ao criar subconjuntos que serão modificados, para evitar `SettingWithCopyWarning`.
- Prefira métodos encadeados (method chaining) para código mais legível.
- Documente as transformações importantes com comentários claros.
- Verifique dtypes e nulos logo no início da análise (`df.info()` e `df.isnull().sum()`).
- Use nomes de colunas sem espaços ou caracteres especiais (ex: `salario_anual` em vez de `'salário anual'`).

17. Script Completo Pronto para Execução

Abaixo está o código completo, corrigido e organizado, que você pode copiar e colar em um Jupyter Notebook ou arquivo `.py`:

```
import pandas as pd
import numpy as np

# === 1. Dados de exemplo ===
dados_funcionarios = {
    'id': [1, 2, 3, 4, 5, 6, 7, 8],
    'nome': ['Ana Silva', 'Bruno Costa', 'Carla Mendes', 'Diego Souza',
            'Elena Rocha', 'Felipe Lima', 'Gabriela Alves', 'Hugo Martins'],
    'idade': [28, 35, 42, np.nan, 31, 29, 45, 38],
    'salario': [4500, 7200, 9800, 5100, 6300, 4800, 11000, 8500],
    'id_departamento': [1, 2, 1, 3, 2, 1, 3, 2],
    'ativos': [True, True, True, False, True, True, True, False]
}
df = pd.DataFrame(dados_funcionarios)

dados_dept = {
    'id_departamento': [1, 2, 3],
    'nome_departamento': ['TI', 'Vendas', 'RH']
}
df_dept = pd.DataFrame(dados_dept)

dados_novos = {
    'id': [9, 10],
    'nome': ['Isabela Freitas', 'João Pereira'],
    'idade': [27, 33],
    'salario': [5200, 6900],
    'id_departamento': [1, 2],
    'ativos': [True, True]
}
df_novos = pd.DataFrame(dados_novos)

# === 2. Exploração básica ===
print('=== DataFrame original ===')
```

```

print(df)
print('\n=== Info ===')
df.info()
print('\n=== Describe ===')
print(df.describe())

# === 3. Seleção e filtros ===
print('\n=== Colunas nome e idade ===')
print(df[['nome', 'idade']])
print('\n=== Idade > 30 e salário > 5000 ===')
print(df[(df['idade'] > 30) & (df['salario'] > 5000)])

# === 4. Nova coluna ===
df['salario_anual'] = df['salario'] * 12
print('\n=== Com salário anual ===')
print(df[['nome', 'salario', 'salario_anual']])

# === 5. Tratamento de nulos e tipos ===
df['idade'] = df['idade'].fillna(df['idade'].mean()).astype(int)
print('\n=== Após preencher nulos e converter idade ===')
print(df[['nome', 'idade']])

# === 6. Value counts e groupby ===
print('\n=== Contagem de ativos ===')
print(df['ativos'].value_counts())
print('\n=== Média salarial por departamento ===')
print(df.groupby('id_departamento')['salario'].mean())

# === 7. Ordenação ===
print('\n=== Ordenado por salário (decrescente) ===')
print(df.sort_values(by='salario', ascending=False)[['nome', 'salario']])

# === 8. Merge ===
df_final = pd.merge(df, df_dept, on='id_departamento')
print('\n=== Após merge com departamentos ===')
print(df_final[['nome', 'nome_departamento', 'salario']])

# === 9. Concat ===
df_todos = pd.concat([df, df_novos], ignore_index=True)
print('\n=== Todos os funcionários (concat) ===')
print(df_todos)

# === 10. Exportação ===
df_todos.to_csv('todos_funcionarios.csv', index=False)
df_todos.to_excel('todos_funcionarios.xlsx', index=False)
print('\nArquivos exportados com sucesso!')

```

18. Conclusão

Este guia cobre as operações essenciais de manipulação de dados com Pandas: carregamento, seleção, filtragem, criação/remoção de colunas, estatísticas descritivas, tratamento de nulos, conversão de tipos, agrupamentos, ordenação, junções (merge) e concatenação.

Com esses fundamentos você está preparado para realizar análises exploratórias robustas e preparar dados para modelagem. Recomenda-se praticar com datasets públicos (ex: Kaggle) e explorar a documentação oficial do Pandas.

Bom aprendizado e boas análises!